

Teoría de la Computabilidad

Módulo 7: Lenguajes sensibles al contexto Lenguajes recursivos

Departamento de Cs. e Ing. de la Computación
Universidad Nacional del Sur
Bahía Blanca, Argentina

Lenguajes recursivos

¿Es este programa en Pascal sintácticamente válido?

```
program FCC;
var a,b:integer;
    c:boolean;
begin
  a:=2;b:=a+3; c:=a>b;
  if c then
    write(a)
  else
    write(b);
end.
```

Lenguajes recursivos

¿Es este programa en Pascal sintácticamente válido?

```
program FCC;
var aux1,aux2,aux3:integer;
    otro_aux:boolean;
begin
  aux1:=0; aux2:=3; aux3:=4;
  if otro_aux then
    otro_aux:= not(otro_aux)
end.
```

Lenguajes recursivos

Hemos realizado recién, aunque en forma intuitiva, parte de la tarea de un compilador de Pascal

El compilador siempre controla inicialmente que nuestro programa sea un programa válido de Pascal

Los lenguajes de programación son lenguajes formales, como los que estudiamos en la materia (aunque mucho más complejos...)

En el caso de Pascal, nosotros ya estamos familiarizados con sus reglas gramaticales y podemos decidir con cierta facilidad si un programa dado pertenece al lenguaje Pascal o no

Lenguajes recursivos: Concepto

Poder realizar esta validación es sumamente importante en las Ciencias de la Computación...

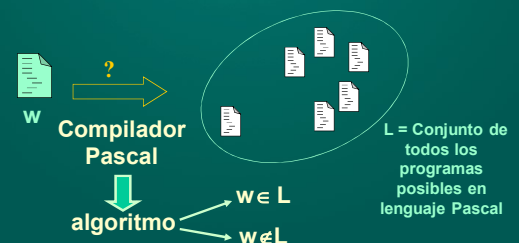
Sea P un lenguaje de programación.
¿Es posible determinar, para todo programa q , si es sintácticamente válido en el lenguaje P ?

O formulado de otra manera...

Sea L un lenguaje formal, $L \subseteq \Sigma^*$
¿Es posible determinar, para toda cadena $w \in \Sigma^*$, si w pertenece a L ?

Lenguaje Recursivo: Concepto

Def.: Sea L un lenguaje formal. Se dirá que L es **recursivo** si existe un algoritmo para determinar, para toda palabra w , si $w \in L$.



Los lenguajes SC son recursivos

Lema: Si G es una gramática sensible al contexto, entonces $L(G)$ es recursivo.

Demostración:

La demostración es constructiva. Sea $G=(V_n, V_t, S, P)$ una gramática sensible al contexto. Entonces

$$\lambda \in L(G) \Leftrightarrow \text{hay una regla } S \rightarrow \lambda.$$

Luego, tenemos un test (algoritmo) para determinar si $\lambda \in L(G)$.

Los lenguajes SC son recursivos

Removamos esa producción, y consideremos

$$G_1=(V_n, V_t, S, P - \{S \rightarrow \lambda\})$$

que genera $L(G) - \{\lambda\}$. Toda regla en $P - \{S \rightarrow \lambda\}$ sigue satisfaciendo las restricciones de una GSC

Sea w una cadena cualquiera. Debemos encontrar un modo de determinar si w pertenece a $L(G)$...

Idea Intuitiva

Observemos la siguiente propiedad de las GSC. Para toda regla de producción $\alpha \rightarrow \beta$, se cumple:

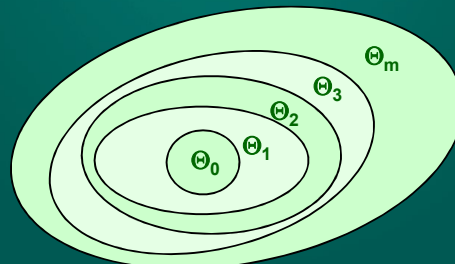
$$\text{long}(\alpha) \leq \text{long}(\beta)$$

Esto significa informalmente, que en el proceso de derivación, al aplicar una regla de producción, obtendremos siempre una cadena de igual o mayor tamaño...

$$\begin{aligned} X_0 &\xrightarrow{G} aX_1bc \xrightarrow{G} abX_1c \xrightarrow{G} abX_2bcc \xrightarrow{G} \\ &\xrightarrow{G} aX_2bbcc \xrightarrow{G} aabbcc \end{aligned}$$

Idea Intuitiva

Entonces, si $|w|=k$, podemos generar fácilmente de forma *exhaustiva* todas las cadenas c tal que $|c| \leq k$.



En algún momento nos daremos cuenta si es imposible generar w

Los lenguajes SC son recursivos

Consideremos P sin $S \rightarrow \lambda$, y sea $w \in V_t^+$, y $\text{long}(w) \leq k$.

Sea Θ_m el cjo. de cadenas $\gamma_1\beta\gamma_2 \in (V_t \cup V_n)^*$ que se pueden derivar de Θ_{m-1} por aplicación de una única regla de producción, y tal que las cadenas tengan longitud $\leq k$. Formalmente:

$$\Theta_0 = \{S\},$$

$$\Theta_m = \Theta_{m-1} \cup \{ \gamma_1\beta\gamma_2 \mid \exists \gamma_1\alpha\gamma_2 \in \Theta_{m-1}, \alpha \xrightarrow{G} \beta \in P, |\gamma_1\beta\gamma_2| \leq k \}$$

Es claro que $\Theta_m \supseteq \Theta_{m-1}$ para todo $m \geq 1$.

Ya que depende sólo de Θ_{m-1} , si en algún momento $\Theta_m = \Theta_{m-1}$ ent. $\Theta_{m-1} = \Theta_m = \Theta_{m+1} \dots$. Es decir, el algoritmo calcula hasta un valor m tal que $\Theta_m = \Theta_{m-1}$

Los lenguajes SC son recursivos

Luego : si $S \xrightarrow{G^+} \gamma_1\beta\gamma_2$ y $|\gamma_1\beta\gamma_2| \leq k$, entonces $\exists \gamma_1\beta\gamma_2 \in \Theta_m$ para algún m .

• Si S no deriva $\gamma_1\beta\gamma_2$ ent. $w = \gamma_1\beta\gamma_2 \notin \Theta_m$

• Si $|\gamma_1\beta\gamma_2| > k$, ent. w no es una cadena sobre la cual estamos considerando pertenencia.

Tenemos entonces un proceso incremental donde se van obteniendo nuevas cadenas según el cjo P . Como el cjo. de reglas a usar es finito, existe una cota finita dependiente de k , y el proceso necesariamente debe terminar (¡algoritmo!).

Se llega a $\Theta_m = \Theta_{m-1}$ cuando todas las cadenas c , $|c| \leq k$ han sido generadas. En virtud de esto, podemos concluir que los LSC son recursivos. CQD

Ejemplo

Sea $G=(V_n, V_t, S, P)$, $V_n = \{S, B, C\}$, $V_t = \{a, b, c\}$ y

$P=\{$
 $S \rightarrow aSBC,$
 $S \rightarrow aBC$
 $CB \rightarrow BC,$
 $aB \rightarrow ab,$
 $bB \rightarrow bb,$
 $bC \rightarrow bc,$
 $cC \rightarrow cc \}$

Queremos ver si $abac \in L(G)$. Construimos para esto los conjuntos $\Theta_1 \dots$

13

Ejemplo

Cadena buscada $w = abac$

$\Theta_0 = \{S\}$

$\Theta_1 = \{S, aSBC, aBC\}$

(usando las 2 primeras reglas)

$\Theta_2 = \{S, aSBC, aBC, abC\}$

(si bien $aaSBCBC$ y $aaBCBC$ pueden derivarse de $aSBC$, éstas no pertenecen a Θ_2 , pues su longitud es $>|w|=4$).

$\Theta_3 = \{S, aSBC, aBC, abC, abc\}$

$\Theta_4 = \Theta_3$, ent. como $abac \notin \Theta_3$ luego $abac \notin L(G)$.

14

Abordando Gramáticas Tipo 0

Veamos que pueden codificarse las gramáticas tipo 0, cuyos símbolos terminales son 0 y 1.

Supongamos que los lenguajes se definen sobre el alfabeto $\Sigma = \{0,1\}$. Esto no supone pérdida de generalidad.

$\Sigma_1 = \{a, b, c, \dots, z\}$

$a = 00000,$
 $b = 00001,$
 $c = 00010,$
 $\dots$
 $z = 11010$

Siempre es posible establecer un mapeo con una secuencia de 0's y 1's.

15

Abordando Gramáticas Tipo 0

Podemos suponer también que disponemos de una cantidad enumerable de *no terminales* $S_0, S_1, S_2 \dots$ de estas gramáticas.

Equivalencia:

$S_0, S_1, S_2 \dots$ $01, 011, 0111, \dots$
 Terminales 0 y 1 00 y $001,$
 $\rightarrow y \text{ ', '}$ 0011 y $00111,$

Ejemplo: las reglas de producción

$\{S_0 \rightarrow 0 S_0 1, S_0 \rightarrow 01\}$

corresponden a la cadena

$01 \ 0011 \ 00 \ 01 \ 001 \ 00111 \ 01 \ 0011 \ 00 \ 001$

$S_0 \rightarrow 0 S_0 1, S_0 \rightarrow 0 1$

16

Abordando Gramáticas Tipo 0

Ahora pueden enumerarse las gramáticas de tipo 1.

Ordenamos las cadenas resultantes por longitud (y aquellas de igual longitud, por su valor numérico), y elegimos entre ellas a medida que aparezcan las que corresponden a GSC.

$000000000..00000011$ GSC_0
 $000000000..00000111$ GSC_1
 $000000001..00000010$ GSC_2
 $000000011..00000111$ GSC_3
 $\dots$
 $000000101..0000101$ GSC_n
 $\dots$

17

Abordando Gramáticas Tipo 0

Aplicando un mecanismo análogo, pueden enumerarse todas las cadenas posibles definibles sobre Σ .

$000..001$ w_0
 $000..010$ w_1
 $001..011$ w_2
 $011..111$ w_3
 $\dots$
 $101..101$ w_n
 $\dots$

18

Intuición : arreglos conteniendo cadenas y gramáticas

Resulta un arreglo **Nat** y otro **GSC**, donde se puede establecer una correspondencia biunívoca (1 a 1).

Nat		GSC	
1	000..001 w_0	1	0000..00011 GSC_0
2	000..010 w_1	2	0000..00111 GSC_1
3	001..011 w_2	3	0001..00010 GSC_2
4	011..111 w_3	4	0011..00111 GSC_3
...	...	...	...
n	101..101 w_n	n	0101..00101 GSC_n
...	...	...	...

Situación resultante

000..001 w_0	0000..00011 GSC_0
000..010 w_1	0000..00111 GSC_1
001..011 w_2	0001..00010 GSC_2
011..111 w_3	0011..00111 GSC_3
...	...
101..101 w_n	0101..00101 GSC_n
...	...

Para una cadena w_i , tenemos “espejada” una gramática G_i . Esto nos interesa solamente para por lo siguiente: siempre podemos saber si $w_i \in G_i$ o no pues G_i es siempre una GSC

Lenguajes recursivos y lenguajes SC

Lema: Existen lenguajes recursivos que no son sensibles al contexto (es decir, no son generados por ninguna gramática tipo 1).

Demostración:

Sea Σ un alfabeto (podemos suponer $\Sigma=\{0,1\}$), y

$$\Sigma^*=\{w_1, w_2, \dots, w_k, \dots\}$$

una enumeración (sin repetición) de todas las cadenas generables a partir de Σ .

Lenguajes recursivos y lenguajes SC

Sea

$$GSC_{\Sigma} = \{G_1, G_2, G_3, \dots\}$$

una enumeración sin repeticiones de las GSC definidas sobre Σ . Sea

$$L' = \{w_i \mid w_i \notin L(G_i)\} \quad [1]$$

Probaremos

- (a) que L' es un lenguaje recursivo, y
- (b) que no puede ser generado por una GSC.

Lenguajes recursivos y lenguajes SC

a) Probemos que L' es recursivo

Dado $w \in \Sigma^*$, por usar la enumeración de Σ^* podemos determinar el índice i para el cual $w_i = w$ (recorriendo el arreglo **Nat**)

Ahora podemos usar la enumeración de GSC_{Σ} para determinar G_i .

Por def. de L' , podemos decidir si $w_i \in L(G_i)$. Si esto sucede, $w_i \notin L'$, caso contrario $w_i \in L'$.

Dado que cada G_i es una GSC, es decidible (por teorema anterior) si w_i está o no en L' .

Luego L' es recursivo.

Lenguajes recursivos y lenguajes SC

000..011 w_0	0000..00011 GSC_0
000..111 w_1	0000..00111 GSC_1
001..010 w_2	0001..00010 GSC_2
011..111 w_3	0011..00111 GSC_3
...	...
101..101 w_i	0101..00101 GSC_i
...	...

¿ $w_i \in L(GSC_i)$?
no $\rightarrow w_i \in L'$

Lenguajes recursivos y lenguajes SC

b) Probemos que L' no es sensible al contexto

Lo haremos por el absurdo. Supongamos que $L' = \{w_i \mid w_i \in L(G_i)\}$ es sensible al contexto.

Luego existe una gramática sensible al contexto G_j que genera L'

Es decir, $L' = L(G_j)$ [2], donde G_j es GSC. Pero G_j estará también en GSC_Σ , pues allí están todas las GSC basadas en Σ . En particular, si consideramos la cadena w_j , se tiene

$$w_j \in L' \iff w_j \in L(G_j), G_j \text{ GSC} \iff w_j \notin L'$$

Esto es un absurdo que provino de suponer que L' era sensible al contexto. **Luego L' no es SC, cqd.**

25

Teorema de clausura de propiedades para LSC

Teorema: La clase de los lenguajes sensible al contexto es cerrada bajo

- unión,
- concatenación,
- estrella de Kleene,
- intersección,
- sustitución libre de cadena vacía, y
- homomorfismo libre de cadena vacía.

26

Propiedades de Clausura: Síntesis

	LR	LLC	LSC	LEF
Unión	✓	✓	✓	✓
Concat.	✓	✓	✓	✓
Est.Kleene	✓	✓	✓	✓
Inter.c/LR	✓	✓	✓	✓
Reversa	✓	✓	✓	✓
Sust.lib.λ	✓	✓	✓	✓
Homom.	✓	✓	✓	✓
Intersec.	✓	X	✓	✓
Complem.	✓	X	✓*	✓

* = Demostrado en 1988 (Teorema de Immerman–Szelepcsényi)

27

Leng. de Prog. Sensibles al Contexto

Hemos visto que las gramáticas libres de contexto **no** permiten generar todos los lenguajes de programación conocidos (por ejemplo, Pascal).

Veremos algunos ejemplos de lenguajes que representan características de lenguajes de programación sensibles al contexto

28

Ejemplo

El lenguaje

$$L_1 = \{wcw \mid w \in \{a,b\}^*\}$$

es sensible al contexto. El problema de reconocer L_1 puede homologarse con el problema de chequear si un identificador es declarado antes de ser usado.

Con esto estamos reiterando que Pascal no es libre de contexto, pues todo identificador debe declararse antes de usarse.

Notemos que la sintaxis de Pascal es usualmente descrita con diagramas sintácticos, que son equivalentes a una gramática libre de contexto.

29

Ejemplo

El lenguaje

$$L_2 = \{a^n b^m c^n d^m \mid n \geq 1, m \geq 1\}$$

es sensible al contexto.

Un ejemplo: aquellos lenguajes que requieren que la declaración de procedimientos esté acompañada de igual número de parámetros formales y de parámetros efectivos (al momento de realizar la invocación).

$a^n b^m$ representaría **parámetros formales**, y $c^n d^m$ representaría **parámetros efectivos**.

30